Instructions

- Complete all questions and test your programs.
- Focus on understanding the logic instead of memorizing syntax.
- You may use AI tools to understand concepts, get hints, or debug your code, but make sure you understand the final solution.
- Try to solve the questions yourself before asking AI for the solution.

Part A – Understand the Concepts

1. Revise **Strings** in Python. Understand how strings are created and stored.
2. Explain the difference between a **single-character string** and a **string containing multiple characters**.
3. Go through **string indexing and slicing** again. Understand how individual characters and portions of a string can be accessed.
4. Revise commonly used string methods such as:
    - upper()
    - lower()
    - strip()
    - replace()
    - split()
    - find()

Understand what each method does and create one example for each.

5. What does it mean when we say that **strings are immutable** in Python? Explain in your own words.
6. Revise **Lists**. Explain:
    - What is a list?
    - How is a list created?
    - Why are lists useful?
7. Go through common list operations such as:
    - Adding elements
    - Removing elements

- Updating elements
- Accessing elements
- Sorting elements

8. Revise **Tuples**. What is a tuple and when would you use one instead of a list?
9. Explain the main difference between a **list and a tuple**.
10. Revise **Sets**. What is a set and what makes it different from a list?
11. Why are duplicate values not normally stored in a set?
12. Understand the basic set operations:

- Union
- Intersection
- Difference

Create your own examples for each.

Part B – Strings

13. Write a program that accepts a person's name and:

- Converts it to uppercase
- Converts it to lowercase
- Displays the number of characters

14. Accept a sentence from the user and count how many words it contains.
15. Accept a string and count how many times a particular character appears in it.
16. Accept a sentence and replace one word with another word.
17. Accept an email address from the user and display:

- Username
- Domain

Example:

student@gmail.com

Output should identify student and gmail.com separately.

18. Write a program that accepts a sentence and removes unnecessary spaces from the beginning and end.

19. Accept a string and check whether a particular word or character exists in it.

Part C – Lists

20. Create a list containing the names of five employees and display the list.
21. Create a list of five numbers and:

- Add a new number
- Remove one number
- Update one number
- Display the final list

22. Create a list of numbers and calculate:

- Sum
- Maximum value
- Minimum value
- Average

23. Create a list containing five product prices. Apply a 10% discount to every product and store the updated prices in another list.
24. Create a list of employee names. Add two new employees and remove one employee from the list.
25. Create a list of cities and check whether a particular city exists in the list.
26. Create a list containing duplicate values. Remove the duplicates and create a new list containing only unique values.

Part D – Tuples

27. Create a tuple containing information about an employee:

- Employee ID
- Name
- Department
- Experience

28. Create a tuple containing five numbers and calculate their:

- Sum

- Maximum
- Minimum

29. Convert the following list into a tuple:

["Python", "SQL", "Excel", "Power BI"]

30. Create a tuple containing the names of five cities. Access and display:

- The first city
- The last city
- The first three cities

31. In your own words, explain why you might choose a tuple instead of a list when working with data that should not be changed.

Part E – Sets

32. Create two sets:

Set A: Employees who know Python
Set B: Employees who know SQL

Find:

- Employees who know both Python and SQL
- Employees who know Python or SQL
- Employees who know Python but not SQL

33. Create a list containing duplicate employee names. Convert it into a set to obtain unique employee names.

34. Create two sets containing students enrolled in two different courses. Find the students enrolled in both courses.

35. Create two sets of numbers and perform:

- Union
- Intersection
- Difference

Part F – Practical Questions

36. You are given the following list of customer names:

["Rahul", "Priya", "Amit", "Rahul", "Sneha", "Amit", "John"]

Create a program that produces a collection containing only unique customer names.

37. A company maintains a list of employee skills:

["Python", "SQL", "Python", "Excel", "SQL", "Power BI"]

Find all the unique skills available in the list.

38. A shopping application stores the following products:

["Laptop", "Mouse", "Keyboard", "Laptop", "Monitor", "Mouse"]

Remove duplicate products and display the unique products.

39. Create a program that accepts five student names from the user and stores them in a list. Then:

- Display all students
- Add another student
- Remove one student
- Display the final list

40. Create a simple **Employee Skills Analysis** program.

Create two sets:

Python Developers:
{"Amit", "Rahul", "Sneha", "Priya"}

SQL Developers:
{"Rahul", "Priya", "John", "Sneha"}

Find:

- Employees who know both Python and SQL
- Employees who know only Python
- Employees who know only SQL
- Total unique employees

Part G – Self-Learning / Revision

41. Go through the difference between **String, List, Tuple, and Set** and create a small comparison table containing:

- Ordered or unordered
- Allows duplicates or not

- Mutable or immutable
- Example use case

42. Explore at least **five useful string methods** that were not discussed in class. Understand what they do and write examples.
43. Explore at least **three additional list methods**. Understand when each one can be useful.
44. Go through the concept of **mutability and immutability** again. Identify which of the four data types learned today are mutable and which are immutable.
45. Think of a real-world application where:

- A **list** would be useful
- A **tuple** would be useful
- A **set** would be useful
- A **string** would be useful

Give one example for each.

⭐ **Challenge**

46. Create a **Student Course Management Program**.

The program should maintain:

- A list of student names
- A tuple containing available courses
- A set containing students enrolled in Python
- A set containing students enrolled in SQL

Your program should allow you to:

- Add a new student
- Remove a student
- Display all students
- Display available courses
- Find students enrolled in both Python and SQL
- Find students enrolled only in Python
- Find students enrolled only in SQL

Try to make the program as user-friendly as possible.